

Worksheet No. - 3

Student Name: Anjali kumari

Branch: MCA general

Semester: 3rd

Subject Name: MACHINE LEARNING.

Subject Code: CONT_25CAP-702

UID: 25MCA20023

Section: Group:1/A

Date of Performance: 5th sept.

Experiment 1 Implement classification algorithms—Logistic Regression and k-NN—on a suitable dataset and evaluate their performance using confusion matrix, precision, recall, and F1-score._

1.Aim:

To implement and compare Logistic Regression and k-Nearest Neighbors (k-NN) classification algorithms on a suitable dataset and evaluate their performance using accuracy, precision, recall, F1-score, and confusion matrix On **Breast cancer dataset**.

2.Objective:

- To understand the concept of **classification algorithms** in machine learning.
- To implement **Logistic Regression** for classification.
- To implement **k-Nearest Neighbors (k-NN)** for classification.
- To evaluate the performance of both algorithms using **accuracy, precision, recall, F1-score, and confusion matrix**.
- To compare the performance of Logistic Regression and k-NN and identify the better-performing model.

3.Task to be done :

- Load and preprocess the **Breast Cancer dataset**.
- Separate the dataset into **input features (X)** and **target variable (y)**.
- Split the data into **training and testing sets** and apply feature scaling.
- Train and test **Logistic Regression and k-NN models** and generate predictions.
- Calculate evaluation metrics, display confusion matrices, and **compare both models** using a performance graph.

4. Online Resources:

1. NumPy Documentation
2. Pandas Documentation
3. Scikit-Learn Official Documentation
4. Matplotlib Official Documentation
5. Seaborn Documentation
6. Kaggle Titanic Dataset Resources

Software Requirements

1. Python 3.x
2. Jupyter Notebook / Google Colab
3. Anaconda Distribution

Standard Tools Used

1. NumPy
2. Pandas
3. Matplotlib
4. Seaborn
5. SciPy
6. Scikit-Learn

5. Procedure/ Algorithm:

```
# =====  
# LOGISTIC REGRESSION  
# =====  
import pandas as pd  
from sklearn.preprocessing import LabelEncoder, StandardScaler  
df = pd.read_csv(  
    r"C:\Users\Dell\Downloads\archive\data.csv"  
)  
  
df = df.drop(  
    columns=["id", "Unnamed: 32"],  
    errors="ignore"  
)  
  
df["diagnosis"] = df["diagnosis"].map({  
    "B": 0,  
    "M": 1  
})  
  
# Separate X and y  
X = df.drop("diagnosis", axis=1)  
y = df["diagnosis"]  
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.25,  
    random_state=10,  
    stratify=y  
)
```

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Create Logistic Regression model
lr_model = LogisticRegression(max_iter=5000)

# Train model
lr_model.fit(X_train_scaled, y_train)
# Prediction
y_pred_lr = lr_model.predict(X_test_scaled)

# Evaluation
accuracy_lr = accuracy_score(y_test, y_pred_lr)
precision_lr = precision_score(y_test, y_pred_lr)
recall_lr = recall_score(y_test, y_pred_lr)
f1_lr = f1_score(y_test, y_pred_lr).....continue,
```

Output:

```
=====
LOGISTIC REGRESSION
=====
Accuracy : 0.972
Precision : 0.9623
Recall : 0.9623
F1-Score : 0.9623
```

```
Classification Report:
      precision    recall  f1-score   support

   Benign         0.98      0.98      0.98        90
  Malignant         0.96      0.96      0.96        53

   accuracy                0.97        143
  macro avg         0.97      0.97      0.97        143
 weighted avg         0.97      0.97      0.97        143
```

```
Confusion Matrix:
[[88  2]
 [ 2 51]]
```

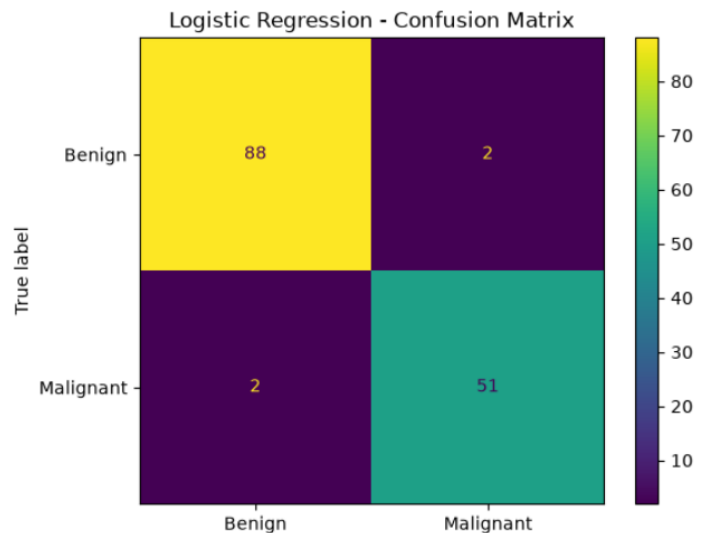


FIG 1(a):Logistic Regression.

FIG 1(b): Graph Representation.

```
# =====
# KNN
# =====
# Load dataset
df = pd.read_csv(r"C:\Users\Dell\Downloads\archive\data.csv")

# Remove unnecessary columns
df = df.drop(columns=["id", "Unnamed: 32"], errors="ignore")
```

```

# Convert diagnosis
df["diagnosis"] = df["diagnosis"].map({"B": 0, "M": 1})

# Input and output
X = df.drop("diagnosis", axis=1)
y = df["diagnosis"]

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.25,
    random_state=10,
    stratify=y
)

# Scaling
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# k-NN model
knn_model = KNeighborsClassifier(n_neighbors=5)

knn_model.fit(X_train_scaled, y_train)

# Prediction
y_pred_knn = knn_model.predict(X_test_scaled).....continue,
  
```

OUTPUT:

```

===== k-NN =====
Accuracy : 0.979
Precision : 1.0
Recall : 0.9434
F1-Score : 0.9709

Confusion Matrix:
[[90  0]
 [ 3 50]]
  
```

FIG 2(a): KNN

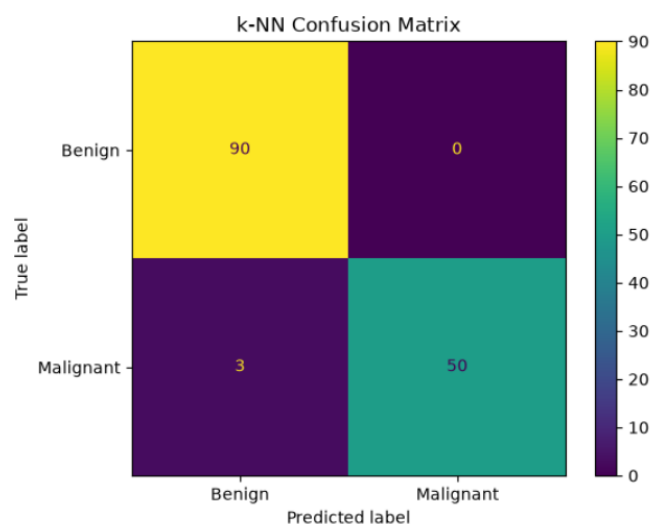


FIG 2(b): Graph Representation

```
# =====  
# COMPARISON  
# =====
```

```
comparison = pd.DataFrame({  
    "Metric": ["Accuracy", "Precision", "Recall", "F1-Score"],  
    "Logistic Regression": [  
        accuracy_lr,  
        precision_lr,  
        recall_lr,  
        f1_lr  
    ],  
  
    "k-NN": [  
        accuracy_knn,  
        precision_knn,  
        recall_knn,  
        f1_knn  
    ]  
})
```

```
print(comparison)
```

```
# Comparison graph  
comparison.set_index("Metric").plot(  
    kind="bar",  
    figsize=(9, 5)  
)
```

```
plt.title("Logistic Regression vs k-NN")  
plt.xlabel("Metrics")  
plt.ylabel("Score")  
plt.ylim(0, 1.1)  
plt.xticks(rotation=0)  
plt.grid(axis="y")  
plt.show()
```

Metric	Logistic Regression	k-NN
0 Accuracy	0.972028	0.979021
1 Precision	0.962264	1.000000
2 Recall	0.962264	0.943396
3 F1-Score	0.962264	0.970874

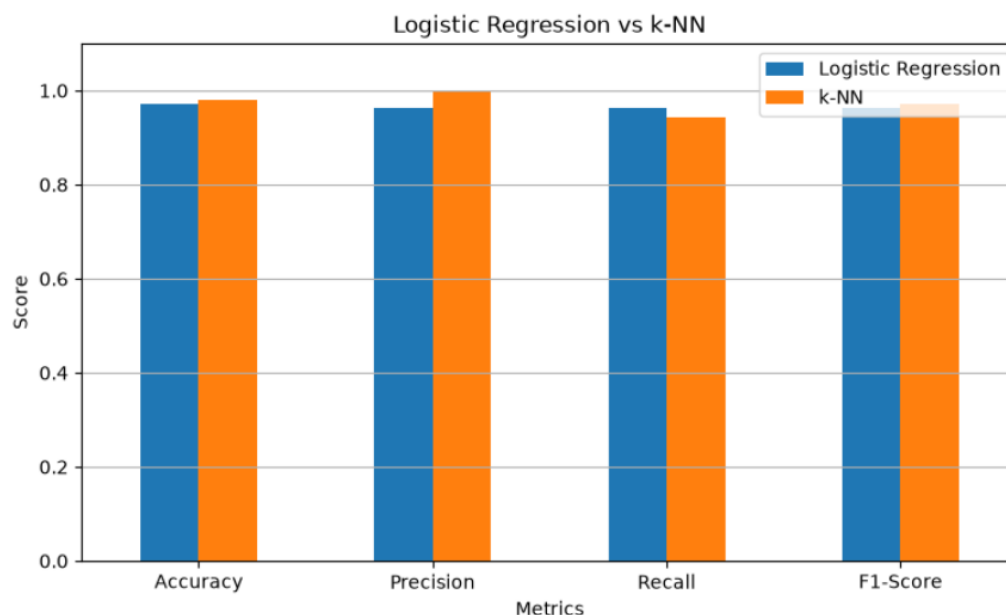


Fig 3: COMPARISON OF LOGISTIC VS KNN .

6. Learning Outcomes

After completing this experiment, students will be able to:

1. Understand the **fundamental concepts of classification in machine learning**.
2. Implement **Logistic Regression and k-NN algorithms** using Python.
3. Perform **data preprocessing, feature scaling, and train-test splitting**.
4. Evaluate classification models using **accuracy, precision, recall, F1-score, and confusion matrix**.
5. **Compare different classification models** and select the better-performing model based on evaluation results.